

Automated TOS & Exam Generator

Multi-PDF Table of Specification and exam builder for teachers.

Laravel 12 + PHP 8.2 + Google Gemini.

Audience: teachers, school admins, and developers evaluating or extending the system.

1. What this project is

A web application that turns a teacher's lesson PDFs and learning outcomes into a balanced Table of Specification (TOS) and a finished exam. The teacher uploads one PDF per lesson (1-15 lessons), states how much class time each lesson received, and pastes in each lesson's learning outcomes. The system classifies every outcome against Bloom's Taxonomy, apportions exam items fairly across lessons and cognitive levels, and then generates the actual questions with AI - each question traceable to its source lesson.

2. Problem it solves

- **Manual TOS tables are slow and error-prone.** Weighting lessons and balancing Bloom's levels by hand takes hours and rarely sums exactly.
- **Single-prompt AI exam generation is uneven.** Stuffing many lessons into one AI call dilutes context and skips lessons. This system gives each lesson its own scoped AI call.
- **No traceability.** Every generated question is tagged with its lesson and Bloom's level, so coverage can be verified.

3. How it works

- **Step 1 - Build the TOS.** Each lesson's outcomes are classified by a built-in machine-learning model. Items are allocated in two levels: first across lessons by teacher weight (class hours), then within each lesson across the six Bloom's levels (20/20/20/15/15/10). Largest-remainder math guarantees counts always sum exactly.
- **Step 2 - Generate the exam.** Each lesson's PDF text is extracted and sent to Google Gemini in parallel - one focused call per lesson. Lower-order levels produce Multiple Choice + Enumeration questions; higher-order levels produce Modified True-or-False + Enumeration. Failed or incomplete items are automatically retried and backfilled.
- **Step 3 - Review and retry.** The teacher reviews the per-lesson and aggregate TOS, then the exam grouped by lesson. Any lesson that failed (e.g. a scanned PDF with no readable text) can be retried individually without touching the rest.

4. Key features

- **Per-lesson pipeline.** Independent classification, quota, and generation per lesson PDF.
- **Exact allocation.** Two-level apportionment - totals always reconcile, with at most ~1 item of aggregate drift.
- **Concurrent AI generation.** Parallel per-lesson calls finish in roughly the time of 1-2 sequential calls.
- **Three question types.** Multiple choice, modified true-or-false (with correction), and enumeration.
- **Single-page interface.** One Blade page; all actions happen via fetch with no reloads. Past TOS records are browsable from a history sidebar.
- **Local Bloom classifier.** Pure-PHP model trained with ``php artisan bloom:train`` - no external ML service needed.

5. Technology stack

- **Backend:** Laravel 12, PHP 8.2, SQLite (default), Eloquent ORM.
- **AI:** Google Gemini (gemini-2.5-flash-lite) via concurrent HTTP pool with retry/backoff.
- **ML:** Logistic-regression Bloom classifier in pure PHP; PDF parsing via smalot/pdfparser.
- **Frontend:** Blade templates, Tailwind CSS v4, Vite; vanilla JS fetch.
- **Tooling:** PHPUnit (SQLite in-memory), Laravel Pint, Pail, Concurrently dev runner.

6. Typical workflow

- **1.** Open `/tos/create`, enter the course name and total exam items.
- **2.** For each lesson: title, weight (class hours), learning outcomes (one per line), and the lesson PDF.
- **3.** Submit, review the TOS balance, then click Generate Exam.
- **4.** Review the exam grouped by lesson; retry any failed lesson individually.

7. Current limitations

- **Scanned PDFs need OCR.** Image-only PDFs yield no text and are skipped; OCR is not built in.
- **No per-lesson Bloom override.** All lessons share the same global Bloom percentage balance.
- **No auth yet.** History is unscoped; user accounts are a future addition.